

EXPERIMENT – 4

Demonstration of Hash Cracking Using John the Ripper Tool in Kali Linux

Aim

To demonstrate password hash cracking using the John the Ripper tool in Kali Linux for understanding password vulnerabilities and security analysis.

Software Requirements

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Security Tool:** John the Ripper
- **System Requirements:**
 - RAM: Minimum 4 GB (8 GB recommended)
 - Disk Space: Minimum 30 GB
 - Processor: Intel/AMD with Virtualization support

Introduction

Password hashes are commonly used to store user credentials securely. However, weak passwords and outdated hashing techniques can be exploited using password-cracking tools. John the Ripper is a popular open-source password auditing and cracking tool used to identify weak passwords by performing dictionary, brute-force, and hybrid attacks. Using Kali Linux in a virtualized environment allows safe and ethical experimentation without affecting real systems.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start the Kali Linux virtual machine.
3. Log in using Kali credentials.

Step 2: Open Terminal

1. Click on **Terminal** from the Kali Linux menu.
2. Ensure John the Ripper is installed:
3. `john --version`

Step 3: Prepare the Hash File

1. Create a text file containing password hashes (example: hash.txt).
2. Store the hash inside the file:
3. `nano hash.txt`

4. Save and exit the file.

Step 4: Run John the Ripper

1. Execute the John the Ripper command:
2. john hash.txt
3. The tool starts cracking the hash using default wordlists.

Step 5: View Cracked Passwords

1. To display cracked passwords:
2. john --show hash.txt

Output

- John the Ripper successfully cracks weak password hashes.
- The cracked password is displayed in plaintext format.

Result

Hash cracking was successfully demonstrated using the John the Ripper tool in Kali Linux. The experiment highlights the importance of using strong passwords and secure hashing algorithms.

OUTPUT:



```
(student@kali)-[~]
└─$ echo -n gupta | md5sum
0029857d24d596d0ab9363f459abc871 -

(student@kali)-[~]
└─$ nano hash.txt

(student@kali)-[~]
└─$ john --format=Raw-MD5 hash.txt
Using default input encoding: UTF-8
Loaded 2 password hashes with no different salts (Raw-MD5 [MD5 256/256 AVX2 8
x3])
Remaining 1 password hash
Warning: no OpenMP support for this hash type, consider --fork=2
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status
Almost done: Processing the remaining buffered candidate passwords, if any.
Proceeding with wordlist:/usr/share/john/password.lst
Proceeding with incremental:ASCII
gupta
1g 0:00:00:01 DONE 3/3 (2026-02-10 15:32) 0.6410g/s 25537Kp/s 25537Kc/s 25537
KC/s gut0k..ge04l
Use the "--show --format=Raw-MD5" options to display all of the cracked passw
ords reliably
Session completed.

(student@kali)-[~]
└─$ echo -n aravind | md5sum
b691c96a0e0e2674df4943221d5b4767 -

(student@kali)-[~]
└─$ nano hash.txt

(student@kali)-[~]
└─$ john --format=Raw-MD5 hash.txt
Using default input encoding: UTF-8
Loaded 3 password hashes with no different salts (Raw-MD5 [MD5 256/256 AVX2 8x3])
Remaining 1 password hash
Warning: no OpenMP support for this hash type, consider --fork=2
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status
Almost done: Processing the remaining buffered candidate passwords, if any.
Proceeding with wordlist:/usr/share/john/password.lst
Proceeding with incremental:ASCII
aravind
1g 0:00:00:36 DONE 3/3 (2026-02-10 15:37) 0.02732g/s 70636Kp/s 70636Kc/s 70636KC/s aravl5..aravr1n
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.

(student@kali)-[~]
└─$
```